AEC1 · Cómo está resuelta

Paradigmas de la Programación · Angel Panadero Rodriguez · Common LISP (SBCL)

Guía para entender y replicar a mano el fichero `angel_panadero_rodriguez_AEC1.lsp`. De cada ejercicio tienes la idea en palabras sencillas, el código y una **traza**: cómo se va ejecutando paso a paso con el ejemplo del enunciado. Las respuestas teóricas completas están en los comentarios del propio fichero.

1. Cómo ejecutarlo

SBCL ya está instalado y en el PATH (abre una terminal nueva para que lo detecte). Desde la carpeta AEC1:

```
sbcl --script angel_panadero_rodriguez_AEC1.lsp
```

Muestra el resultado de cada ejemplo y, en el ejercicio 9, te pide un número. Para probar funciones sueltas, abre `sbcl` y haz `(load "angel_panadero_rodriguez_AEC1.lsp")`. Sales con `(exit)`.

2. El LISP que necesitas

En LISP todo va entre paréntesis y la función va primero: `(+ 1 2)` es 1 + 2. Con esto tienes todo lo que usa el fichero:

`(defun f (a b) cuerpo)`	Define una función. Devuelve el valor de la última expresión.
`(car l) / (cdr l)`	Primer elemento / resto de la lista: (car '(1 2 3)) → 1, (cdr '(1 2 3)) → (2 3).
`(cons x l)`	Lista nueva con x delante: (cons 1 '(2 3)) → (1 2 3).
`(list a b)`	Crea una lista. La uso como «tupla»: (list 1 2) → (1 2).
`(null l)`	¿Está vacía? Es la condición de los casos base.
`(if c si no)`	Si c es cierto, hace «si»; si no, hace «no».
`(when c a b)`	Como un if sin «no». Permite hacer varias cosas seguidas (a y luego b).
`(min a b) / (max a b)`	El menor / el mayor de dos números: (min 9 3) → 3.
`#'f y (funcall f x)`	Pasar una función como dato / llamar a una función que he recibido.
`(lambda (x) (* 2 x))`	Función sin nombre: «dado x, devuelve x por 2».
`(mapcar f l)`	Aplica f a cada elemento y devuelve una lista nueva (el map de LISP).
`(reduce f l)`	Junta todos los elementos en un valor: (reduce #'+ '(3 2 4)) → 9.
`(format t "texto~%")`	Imprime en pantalla; ~% es salto de línea.
`'(1 2 3) / #\a`	Lista escrita tal cual (el ' evita que se ejecute) / el carácter a.

3. La plantilla que se repite

Casi todos los ejercicios son recursión sobre listas: resuelves el primer elemento (`car`) y llamas a la misma función con el resto (`cdr`) hasta llegar al caso base. Antes de escribir, responde a dos preguntas: **¿cuándo paro y qué devuelvo?** (caso base) y **¿cómo junto el primer elemento con el resultado del resto?** (caso recursivo).

```
(defun f (lista)
  (if (null lista)                          ; ¿paro?
      valor-caso-base                       ; caso base
      (juntar (car lista) (f (cdr lista)))))  ; caso recursivo
```

Truco de lectura: `(null (cdr lista))` significa «el resto está vacío», es decir, **solo queda un elemento**. Así queda cada ejercicio con esta plantilla:

Ejercicio	¿Cuándo paro?	Si no, ...
`2 longitud_lista`	lista vacía → 0	1 + longitud del resto
`3 encontrar_minimo`	un elemento → ese	min(primero, mínimo del resto)
`4 posicion_de`	el primero es el buscado → 1	1 + posición en el resto
`5 encontrar_extremo`	un elemento → ese	funcion(primero, extremo del resto)
`6, 7 obtener_..._n`	n = 1 → el primero	buscar la posición n-1 en el resto
`8a duplicar_lista`	lista vacía → nil	cons(2·primero, resto duplicado)
`9 repetir_accion_n`	n = 0 → no hago nada	hago la acción y repito con n-1

4. Ejercicio a ejercicio

Ejercicio 1 · imprimir_bienvenida

Un `format t` con el texto y `~%` al final. Es la única función que hace algo «hacia fuera» (escribe en pantalla); las demás solo calculan y devuelven un valor: son **funciones puras**.

```
(defun imprimir_bienvenida ()
  (format t "¡Bienvenidos al Paradigma Funcional: Inmutabilidad y Funciones Puras!~%"))
```

Ejercicio 2 · longitud_lista

Si la lista está vacía, mide 0. Si no, mide 1 (el primer elemento) + lo que mida el resto. No mira qué hay en la lista, así que vale para listas de cualquier tipo.

```
(defun longitud_lista (lista)
  (if (null lista)
      0
      (+ 1 (longitud_lista (cdr lista)))))
```

```
; (longitud_lista '(4 8 15))
; = 1 + (longitud_lista '(8 15))
; = 1 + 1 + (longitud_lista '(15))
; = 1 + 1 + 1 + (longitud_lista '())
; = 1 + 1 + 1 + 0 = 3
```

Ejercicio 3 · encontrar_minimo

Si solo queda un elemento, ese es el mínimo. Si no, me quedo con el menor entre el primero y el mínimo del resto, usando `min`. Ojo: `min` solo compara dos números; quien recorre la lista es la recursión.

```
(defun encontrar_minimo (lista)
  (if (null (cdr lista))
      (car lista)
      (min (car lista) (encontrar_minimo (cdr lista)))))
```

```
; (encontrar_minimo '(9 3 7))
; = (min 9 (encontrar_minimo '(3 7)))
; = (min 9 (min 3 (encontrar_minimo '(7))))
; = (min 9 (min 3 7))
; = (min 9 3) = 3
```

Ejercicio 4 · encontrar_minimo_y_posicion

Lo parto en dos problemas: **1)** el mínimo, que ya lo calcula `encontrar_minimo` (ejercicio 3); **2)** en qué posición está, con una función auxiliar `posicion_de`. Al final junto los dos valores con `list`, que hace de «tupla».

`posicion_de` es casi igual que `longitud_lista`: suma 1 por cada elemento que pasa hasta encontrar el que busca. Si el mínimo está repetido, devuelve la primera aparición, porque para en cuanto lo encuentra. Llamar dos veces a `encontrar_minimo` no es problema: como es pura, las dos veces da lo mismo.

```
(defun posicion_de (elemento lista)
  (if (= elemento (car lista))
      1
      (+ 1 (posicion_de elemento (cdr lista)))))

(defun encontrar_minimo_y_posicion (lista)
  (list (encontrar_minimo lista)
        (posicion_de (encontrar_minimo lista) lista)))
```

```
; (encontrar_minimo_y_posicion '(9 1 7 3))
; 1) mínimo:   (encontrar_minimo '(9 1 7 3)) = 1
; 2) posición: (posicion_de 1 '(9 1 7 3))     9 no es 1 -> 1 + ...
;            = 1 + (posicion_de 1 '(1 7 3))   1 es 1    -> 1
;            = 1 + 1 = 2
; resultado:   (list 1 2) = (1 2)
```

Ejercicio 5 · sumar_max_y_min (refactorización)

Parto de `encontrar_minimo`: el máximo se calcularía igual, cambiando `min` por `max`. Así que la copio, la renombro a `encontrar_extremo`, le añado un parámetro `funcion` y cambio `min` por `(funcall funcion ...)`. Luego la llamo con `#'max` y con `#'min` y sumo. Lo único que cambia respecto al ejercicio 3:

```
ej. 3:  (min            (car lista) (encontrar_minimo  (cdr lista)))
ej. 5:  (funcall funcion (car lista) (encontrar_extremo (cdr lista) funcion))
```

```
(defun encontrar_extremo (lista funcion)
  (if (null (cdr lista))
      (car lista)
      (funcall funcion (car lista) (encontrar_extremo (cdr lista) funcion))))

(defun sumar_max_y_min (lista)
  (+ (encontrar_extremo lista #'max)
     (encontrar_extremo lista #'min)))
```

```
; (sumar_max_y_min '(2 6 10)) = (+ 10 2) = 12
```

Ejercicio 6 · obtener_caracter_n

Avanzo por la lista restando 1 a `n`. Cuando `n` vale 1, el primer elemento es el que busco. Si `n` es mayor que la lista, al final queda `(car nil)`, que es `nil`. Los caracteres se escriben `#\a`.

```
(defun obtener_caracter_n (lista n)
  (if (= n 1)
      (car lista)
      (obtener_caracter_n (cdr lista) (- n 1))))
```

```
; (obtener_caracter_n '(#\a #\b #\c #\d) 2)
; = (obtener_caracter_n '(#\b #\c #\d) 1)     n = 1 -> devuelvo el primero
; = #\b
```

Ejercicio 7 · obtener_trinomio_n

Es el mismo código que el 6, cambiando solo el nombre. Cada trinomio es una lista `(1 2 3)`. Ojo: el ejemplo del enunciado pone `obtener_caracter_n` por error; lo indico en un comentario.

¿Se puede generalizar a cualquier tipo? Sí: la función nunca mira qué hay dentro de cada elemento (solo usa `cdr` y `car`) y LISP tiene tipado dinámico, así que vale para números, caracteres, cadenas, trinomios... (polimorfismo). En el fichero lo pruebo con una lista de cadenas.

```
(defun obtener_trinomio_n (lista n)
  (if (= n 1)
      (car lista)
      (obtener_trinomio_n (cdr lista) (- n 1))))
```

```
; (obtener_trinomio_n '((1 2 3) (4 5 6)) 2)        => (4 5 6)
; (obtener_trinomio_n '("uno" "dos" "tres") 3)     => "tres"
```

Ejercicio 8 · duplicar_lista

a) Recursiva: el doble del primero, pegado con `cons` delante del resto ya duplicado. **b)** `mapcar` con una lambda que duplica un número. Como no puede haber dos funciones con el mismo nombre, la b) se llama `duplicar_lista_map`. En los dos casos se crea una lista nueva y la original no cambia.

```
(defun duplicar_lista (lista)
  (if (null lista)
      nil
      (cons (* 2 (car lista)) (duplicar_lista (cdr lista)))))

(defun duplicar_lista_map (lista)
  (mapcar (lambda (x) (* 2 x)) lista))
```

```
; (duplicar_lista '(1 2 3))
; = (cons 2 (cons 4 (cons 6 nil))) = (2 4 6)
```

c) Una lambda es una función sin nombre que se escribe justo donde se usa. Ventajas: se pasa directamente a funciones como `mapcar` o `reduce`, evita crear con `defun` funciones de un solo uso, puede usar variables del sitio donde se escribe (clausura) y el código dice *qué* hacer, no *cómo* recorrer la lista.

Ejercicio 9 · repetir_accion_n

Si `n > 0`: ejecuto la acción con `funcall` y me llamo con `n-1`. Cuando `n` llega a 0, el `when` no hace nada y se para. Uso `when` en vez de `if` porque tengo que hacer dos cosas seguidas.

La demostración va directamente en el fichero (sin función): pregunta un número, lo lee con `(read)` y pasa la bienvenida con `#'imprimir_bienvenida`. `finish-output` hace que la pregunta salga en pantalla antes de esperar la respuesta.

```
(defun repetir_accion_n (accion n)
  (when (> n 0)
    (funcall accion)
    (repetir_accion_n accion (- n 1))))

(format t "~%¿Cuántas veces quieres mostrar la bienvenida? ")
(finish-output)
(repetir_accion_n #'imprimir_bienvenida (read))
```

```
; con n = 2:  n=2 -> imprime   n=1 -> imprime   n=0 -> para
```

Ejercicio 10 · map/reduce y el lenguaje

Incluyo un ejemplo pequeño: cada sublista es un bloque de datos que estaría en un nodo; `mapcar` cuenta cada bloque (con `longitud_lista`) y `reduce` suma esos resultados.

```
(defun contar_elementos_map_reduce (bloques)
  (reduce #'+ (mapcar #'longitud_lista bloques)))
```

```
; (mapcar #'longitud_lista '((1 2 3) (4 5) (6 7 8 9)))  -> (3 2 4)
; (reduce #'+ '(3 2 4))                                  -> 9
```

Ideas clave de las respuestas (desarrolladas en el fichero):

- **a)** Un for va dato a dato y cambia una variable (el acumulador), así que tiene que hacerse en orden y en un solo sitio. Con `map` cada dato se procesa por separado, así que se puede repartir entre muchos nodos a la vez; `reduce` junta los resultados parciales. Ventajas: escala, no hay variables compartidas (ni conflictos entre nodos), si un nodo falla se repite su parte en otro con el mismo resultado, y el código dice *qué* hacer, no *cómo*.
- **b)** Common LISP: multiparadigma con núcleo funcional (funciones como dato, lambdas, recursión, `mapcar`/`reduce`), todo entre paréntesis con la función delante, código y datos son listas (homoiconicidad), memoria automática y REPL. Tipado **dinámico** (los tipos se comprueban al ejecutar) y **fuerte** (`(+ 1 "a")` da error). Implica más flexibilidad (una función vale para cualquier tipo), a cambio de descubrir los errores de tipo al ejecutar.
- **Por qué los lenguajes modernos lo incorporan:** ordenadores con varios núcleos y sistemas distribuidos (sin datos que cambien, el paralelismo es más seguro), código más corto con map/filter/reduce y lambdas, funciones puras fáciles de probar, y encaja con big data.

5. Antes de entregar

- Nombre del fichero exacto: `angel_panadero_rodriguez_AEC1.lsp`.
- Cabecera con `; Paradigmas de la Programación`, `; AEC1` y `; Nombre: ...`, y un comentario `; Ejercicio N` antes de cada ejercicio.
- Nombres de funciones idénticos a los del enunciado, con guion bajo. SBCL los muestra en mayúsculas, pero LISP no distingue mayúsculas de minúsculas.
- Respuestas a las preguntas (7, 8c, 10a, 10b) como comentarios dentro del fichero.
- Ejecútalo una vez con `sbcl --script` y comprueba que no da errores.